AI Compiler Performance Engineering

Auto-Loops Books — ch01

Contents

1 LLM Inference Performance Bottlenecks: From Symptoms to Root Causes **1**

1.1 Why Decode Performance Is the Industry Bottleneck 1
1.2 The AI Compiler Performance Engineer 4
1.3 Prefill and Decode: Two Performance Regimes 5
1.4 Three Core Pain Points of Decode 6
1.5 Why Mainstream Frameworks Do Not Solve Decode Alone 10
1.6 Measuring Decode Goodput 11
1.7 Two-Step Bottleneck Diagnosis 12
1.8 Multi-Hardware Pain Points: GPU, CPU, and NPU 13
1.9 Industry Baselines and What They Still Leave on the Table 14
1.10 The YiRage Answer: Compiler-Scale Fusion (Preview) 16
1.11 Profiling Decode Without Foolish Optimism 16
1.12 Book Roadmap and How to Read 17
1.13 Key Takeaways 19
1.14 Conclusion 20

Bibliography **23**

Index **25**

Chapter 1

LLM Inference Performance Bottlenecks: From Symptoms to Root Causes

1.1 Why Decode Performance Is the Industry Bottleneck

In early 2025, DeepSeek-V3 reset what the market thought frontier models should cost to serve: open weights, aggressive quantization, and a production layout that provisions ten times more decode silicon than prefill silicon—320 GPUs in the minimum decode unit versus 32 in the minimum prefill unit (Davies *et al.*, 2025). The business story was not "more FLOPs"; it was *where* bytes and launches go during autoregressive generation, and how fleet topology must follow that asymmetry.

Export controls and regional hardware diversity add another wrinkle: the "best" SKU in a benchmark may be unavailable in your deployment region, so normalized counters matter more than absolute milliseconds when comparing nodes (Davies *et al.*, 2025; Wang, 2025b). Performance engineering that survives export constraints is exactly why this book teaches triplet benchmarking (GPU, CPU, NPU) rather than single-vendor hero numbers.

That pattern repeats outside headline models. A typical production team we see in performance postmortems (composite of several published serving studies, not one named customer) spends calendar quarters enabling FlashAttention-2, adopting vLLM, and upgrading to H100—yet interactive chat still misses p99

time-to-next-token SLOs because each new token still pays hundreds to thousands of kernel launches and full weight+KV traffic (Vellaisamy *et al.*, 2026; Chen, 2026a; Fan, 2025). The failure mode is familiar: prefill dashboards look excellent; decode latency flatlines. Industry effort skews toward prompt throughput benchmarks, while user experience and recurring inference *opex* concentrate in the decode loop once the prompt is ingested (Wang, 2025b; Li, 2024).

DeepSeek-scale lesson (export-aware). DeepSeek-V3's public inference story combines aggressive weight compression, disaggregated prefill/decode fleets, and hardware layouts tuned to their own network and SKU availability (Davies *et al.*, 2025). Readers in export-controlled regions may not have access to the same GPU generations assumed in US-centric benchmarks. That is why this book stresses **normalized counters** and triplet backends: you can still compare mechanisms when absolute milliseconds or SKU names differ (Wang, 2025b; Chen and YiRage Contributors, 2026). The business lesson remains: decode footprint dominates opex once products scale—not because prompts are short, but because outputs are long and serial (Li, 2024).

Skillful engineering vs. brute-force spend. Fregly's training-goodput narrative applies unchanged to decode: prefer measurement, fusion, and scheduling discipline over buying another rack of accelerators without changing the software schedule (Davies *et al.*, 2025). A mid-range GPU with fused decode can beat a flagship GPU running an eager loop—Memory-Floor's L4 ExLlamaV2 versus H100 CUDA Graphs cell is a concrete example (Chen, 2026a). Your CFO cares about that comparison; your worksheet makes it reproducible.

The takeaway at scale: decode performance engineering is a **goodput** problem—useful tokens delivered per dollar and per millisecond under real batching policies—not a peak-TFLOPs procurement exercise (Recasens, 2025). Single-kernel wins (flash attention, paging, graphs) are necessary but do not, by themselves, collapse the operator boundaries that export activations to HBM on every decoder step (Dao *et al.*, 2022; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). This book argues that compiler-scale, hardware-aware fusion is the remaining lever once serving frameworks and attention libraries are already enabled.

Mechanical sympathy for decode. Performance engineering here is not "make kernels faster" in isolation—it is aligning software schedule with how silicon actually moves bytes. NVIDIA GPUs expose HBM (high-bandwidth memory) tiers reached through SIMT warps; CPUs expose cache lines and NUMA domains; XDNA-class

NPUs expose fixed SRAM slots filled by DMA engines. A schedule that ignores those shapes looks correct in PyTorch and still loses to a smaller card with fewer launches (Chen, 2026a; AMD, 2024b; Recasens, 2025). Chapter 1 names the counters; later chapters name the mechanisms (TMA, ADF, MegaKernel) that move those counters.

What this chapter gives you. This chapter is the book's **performance diagnosis foundation**. You will leave with a shared vocabulary—prefill versus decode, four goodput counters, and a two-step worksheet—so you can tell whether a workload is orchestration-limited, bandwidth-limited, or mis-measured before touching tensor-core tile sizes. That diagnostic discipline carries through kernel craft (Parts IV–V), compiler passes (Part VI), fleet tuning (Part VII), and framework/runtime co-design (Part VIII).

Reader prerequisites and 60-second review. We assume you have used transformer inference in PyTorch or Hugging Face, know that **KV cache** stores prior keys/values so each new token does not recompute the full prompt, and have seen a CUDA "kernel launch" at least once. If any of those are new, skim a vLLM architecture note first (Kwon *et al.*, 2023). Three ideas recur in every section: **(i) Prefill** ingests the prompt in parallel; **(ii) decode** appends one token per forward pass in chat; **(iii) operator fragmentation** means each sub-layer (norm, QKV, attention, MLP) may be a separate kernel that reads and writes global memory (GPU HBM or CPU DRAM) between steps.

Operator fragmentation (warehouse analogy). Picture a factory that must complete norm → QKV → attention → MLP for each token. An eager framework sends the partial product back to the warehouse (HBM) after every station because each station is a different vendor (kernel). Fusion is the engineering decision to keep the assembly line on one bench (registers, shared memory, or cache) until the layer completes (Chen and YiRage Contributors, 2026; Chen, 2020). MegaKernel fusion is not mysticism—it is refusing to ship intermediates to the warehouse when the bench has capacity (Dao *et al.*, 2022).

Why compiler ownership matters now. Frameworks optimize for research agility: new attention variants ship as Python modules (Dao *et al.*, 2022). Production decode rewards the opposite trade—fewer boundaries, static residency, schedules chosen when the graph is known (Chen, 2020; Chen and YiRage Contributors, 2026). The symptoms in this chapter are not PyTorch bugs; they are

consequences of deferring dataflow decisions to a host loop (Vellaisamy *et al.*, 2026; Chen, 2026a). YiRage makes those decisions explicit in IR, verifiable against hardware probes, and regressable before codegen (Chen and YiRage Contributors, 2026).

Reading order for impatient experts. If you already know CUDA: skim §1.3–§1.4, then dive into §1.6 and §1.7. If you are a serving engineer new to kernels: read §1.1 and §1.2, then §1.5 before any roofline math. Everyone should read §1.12 once to see where triplet compilation lands (Chen and YiRage Contributors, 2026).

1.2 The AI Compiler Performance Engineer

This book is written for engineers who sit at the intersection of three disciplines: **inference serving**, **kernel/hardware craft**, and **compiler automation**. You do not need to be all three on day one, but decode optimization fails when any leg is missing. Serving experts without kernel depth propose batching changes that hide but do not remove launch storms. Kernel experts without compiler context ship hand-tuned CUDA that cannot port to the next SKU or to the NPU your product team now requires. Compiler experts without serving empathy optimize graphs that frameworks cannot load or that break sampling and dynamic shapes in production.

Your job in the decode path is to increase **goodput**: useful tokens per second per dollar at the batching policy your product actually ships (Davies *et al.*, 2025; Fan, 2025). That is rarely the same as maximizing TFLOPs or winning a prefill leaderboard. It *is* the same as moving kernels/token, bytes/token, HDBI, and R_{floor} in the right direction on a frozen configuration cell.

How this role shows up in reviews. A credible decode optimization review answers four questions before showing a diff: (1) Which phase—prefill or decode—misses the SLO? (2) Which counter binds on that phase? (3) Which stack layer owns the fix (framework, attention kernel, fusion/compiler)? (4) What open baseline already includes the change? If the review jumps straight to tile sizes, send it back to Chapter 1 (Vellaisamy *et al.*, 2026; Chen, 2026a).

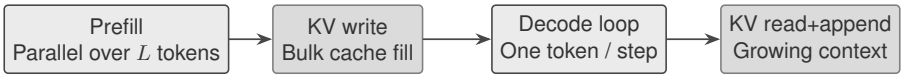


Figure 1.1: Prefill is bulk parallel ingest; decode is a serial per-token loop over growing KV state.

Table 1.1: Prefill vs. decode on one transformer layer (engineering view).

	Prefill	**Decode (batch-1 chat)**
Work shape	L tokens in parallel	1 new token per step
What limits you	Compute / tensor throughput	HBM/DRAM traffic + host la
What users feel	Prompt ingestion time	ms/token, streaming smoothn
Fleet metric	Prompt tok/s	p99 latency, cost per 1M toke

1.3 Prefill and Decode: Two Performance Regimes

Think of prefill as *bulk unloading a truck*: many tokens move together, matrix multiplications are wide, and tensor cores can approach peak throughput (Pichlmeier, 2024; SemiAnalysis, 2024). Decode is *hand delivery one package at a time*: each step touches the full parameter set and an ever-growing KV history while doing only a thin slice of arithmetic (Li, 2024; Eto, 2025). Teams that optimize only the truck unload wonder why doorstep latency never improves.

Production fleets increasingly *disaggregate* prefill and decode pools because tuning one phase can harm the other (Qiao, 2026; Chen, 2026b). DeepSeek-V3's 10:1 decode-to-prefill node ratio is the architectural admission that user-visible cost follows decode steps, not prompt GEMMs alone (Davies *et al.*, 2025).

Industry analyses align. Precision-roofline studies, memory-gap characterizations, and accelerator limit studies independently report that decode at low batch is not compute-starved (Yu, 2026; Recasens, 2025; Davies *et al.*, 2025). The engineering conclusion is consistent even when the hardware generation changes: fix data movement and dispatch, then argue about FLOPs.

Practice note. If your dashboards report only aggregate tok/s, split them tomorrow: one panel for prompt ingestion, one for batch-1 decode on the same model revision.

After countless production postmortems, teams converge on the same conclusion: interactive decode is typically memory-bandwidth and orchestration-limited on

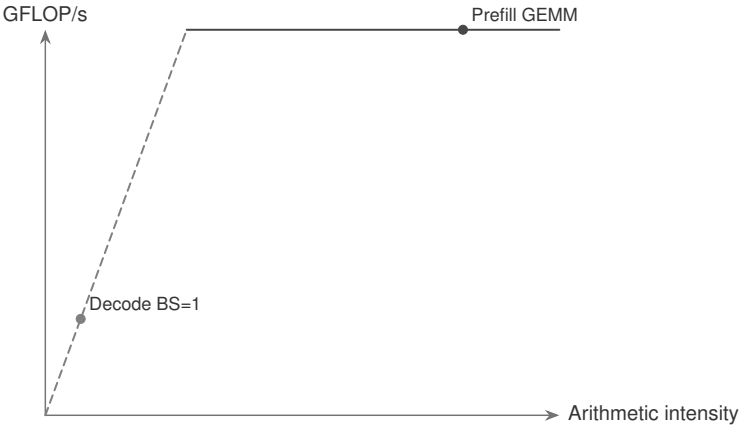


Figure 1.2: Roofline intuition: decode batch-1 sits on the bandwidth slope; prefill wide GEMMs reach the compute cap.

GPUs, CPUs, and many NPUs—not FLOP-limited (Yu, 2026; Recasens, 2025; Davies *et al.*, 2025). Treating prefill and decode as one optimization playbook—bigger batches, FlashAttention defaults, tensor-core-first schedules—is why decode SLOs stay red after prefill tuning succeeds (Tang, 2026; Dao *et al.*, 2023).

1.4 Three Core Pain Points of Decode

Interactive assistants, copilots, and voice interfaces live or die on *time-to-next-token*, usually at batch size one (AMD, 2024a; Fan, 2025). After working through dozens of decode postmortems, we group failures into three recurring pain points—not because the list is exhaustive, but because it drives the diagnostic worksheet in §1.7.

1.4.1 Pain point 1: The memory wall (bytes per token)

At batch-1, each layer performs narrow GEMMs while *reading* every weight byte and every KV byte again. Arithmetic intensity (FLOPs per byte moved) sits on the bandwidth-limited region of the roofline, not the flat compute cap (Yu, 2026; Recasens, 2025; Davies *et al.*, 2025).

Memory-Floor quantifies how close you are to the bandwidth roof with $R_{\text{floor}} = t_{\text{floor}}/t_{\text{obs}}$ where $t_{\text{floor}} = (W + K)/B_{\text{peak}}$ (Chen, 2026a). On Qwen-2.5-7B at ctx=2048, batch-1: L4 reaches R_{floor}=0.81 while H100 reaches 0.27—faster silicon

can be *more* launch-limited, not less.

Engineering read. When R_{floor} is high (≈0.7+), shaving bytes/token (quantized GEMMs, fusion) moves ms/token. When it is low on a flagship GPU, fix launches and fusion before buying another attention kernel.

Roofline arithmetic for decoder layers. Attainable FLOP/s is min(peak FLOP/s, bandwi AI) where arithmetic intensity AI is FLOPs per byte moved. A crude batch-1 model reads N_{params} weight bytes and N_{KV} KV bytes per layer per token, giving

$$\text{AI} \approx \frac{2N_{\text{params}} + 2N_{\text{KV}}}{N_{\text{params}} + N_{\text{KV}}} \tag{1.1}$$

under simplified GEMM accounting (one multiply–add per byte read; activations and writebacks omitted) (Lazuka, 2024; Davies *et al.*, 2025). Context growth raises AI slowly but usually leaves decode on the bandwidth slope for practical LLMs.

1.4.2 Pain point 2: The scheduling tax (kernels per token)

Operator fragmentation is the factory line that hauls every intermediate back to the warehouse between stations: layer norm, QKV, attention, MLP, and residuals become separate kernels, each exporting activations to global memory.

TaxBreak (Vellaisamy *et al.*, 2026) splits host-visible latency per launch into framework translation (ΔFT), CUDA-library front-end (ΔCT), and launch-path floor (ΔKT). Their Host–Device Balance Index (HDBI) compares device-active time to orchestration time; HDBI $\to 0$ means the CPU is the boss and the GPU waits.

On H200, BS=1, SL=512, Llama-3.2-1B issues ≈844 launches per decode step versus ≈85 per prefill step scaled to the same window—an order-of-magnitude dispatch gap (Vellaisamy *et al.*, 2026). At H100's measured null-kernel floor of 4.707 μs, a *single-stream* upper bound of 9,305 launches/token (OLMoE) is ≈44 ms of launch-path time alone—before framework tax and before any math. Concurrent streams and CUDA Graphs overlap some of that cost, but the ≈44 ms figure already crosses the ≈50 ms/token interactive SLA line many products publish (Vellaisamy *et al.*, 2026; Chen, 2026a). **Practice implication:** micro-optimizing one GEMM inner loop cannot recover ms/token when dispatch is already at SLA by itself.

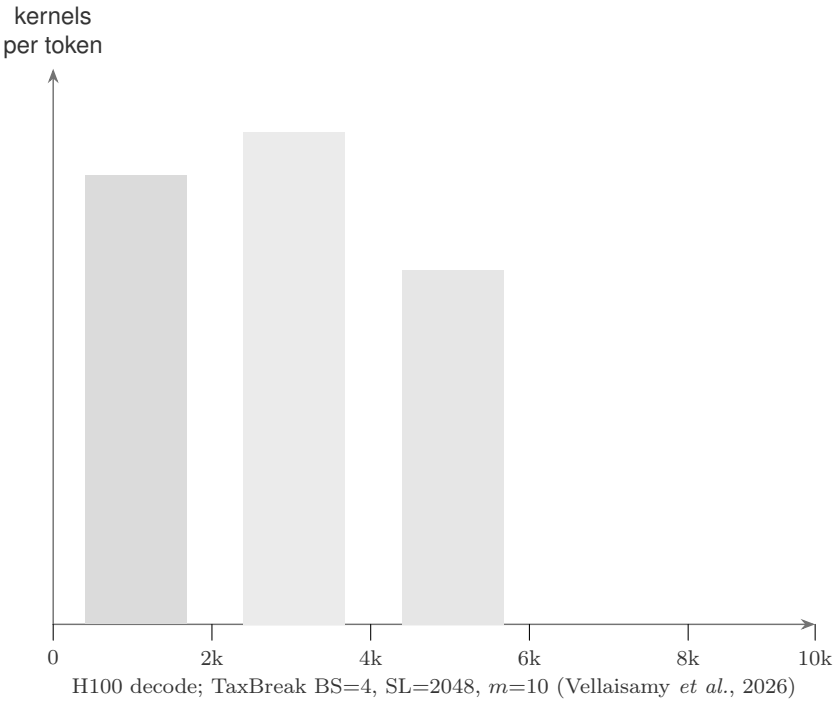


Figure 1.3: Kernel launches per output token: MoE stacks amplify dispatch cost (≈9.3k launches/token for OLMoE).

1.4.3 Pain point 2b: MoE dispatch storms

Mixture-of-experts (MoE) models route each token through a subset of experts, fanning execution into many small GEMMs and auxiliary kernels per step (Vellaisamy *et al.*, 2026; Wang, 2025a). TaxBreak's OLMoE cell shows ≈9.3k launches per output token and GPU utilization near 15%—the device waits on dispatch, not FLOPs (Vellaisamy *et al.*, 2026). Dense-model intuitions like "raise batch size to feed tensor cores" help less when activated experts per token are few and each expert forward is short. Compiler and runtime strategy should prioritize expert fusion, routing caches, and graph capture across the routing boundary before micro-tuning individual expert matmuls (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Part VII returns to MoE fleet scheduling; Chapter 1's lesson is that *kernels/token* can be an order of magnitude higher than dense baselines on the same silicon.

1.4.4 Pain point 3: Fragmentation across the stack

On CPUs the warehouse metaphor becomes last-level cache (LLC) pollution: each unfused boundary evicts lines a fused decoder block could keep resident across norm–matmul–activation chains. On NPUs, fragmentation is an architectural fault: if the compiler cannot prove tensor lifetimes fit on-chip SRAM, bring-up fails rather than degrading gracefully to DRAM-heavy eager paths (AMD, 2024b; Chen and YiRage Contributors, 2026).

A dense decoder block often spans 10–20+ discrete kernels in eager PyTorch; MoE multiplies that count (Vellaisamy *et al.*, 2026). PagedAttention and continuous batching (vLLM) fix *fleet* capacity and KV fragmentation; FlashAttention and Flash-Decoding fix *attention IO*—neither removes per-layer boundaries inside the forward pass (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023). Layer-scale **MegaKernel** fusion—*whole-decoder fusion* in one dataflow-planned kernel group—targets the gaps between those layers (Chen and YiRage Contributors, 2026; Chen, 2020).

Prefill versus decode spend (planning heuristic). Capacity planners often ask what fraction of GPU-hours decode consumes once a product reaches steady state. Published large-model layouts suggest decode can dominate fleet footprint even when prompts are long—the 10:1 DeepSeek decode-to-prefill node ratio is an extreme but directionally common admission that autoregressive steps multiply by output length while prefill runs once per prompt (Davies *et al.*, 2025; Wang, 2025b). If your internal telemetry shows the opposite, verify you are not attributing continuous-batching throughput to per-user decode latency (Kwon *et al.*, 2023; Chen, 2026b).

Context-length stress. At ctx=2048 versus ctx=8192, KV bytes per token grow with history length; Flash-Decoding parallelizes KV dimension work so batch-1 decode can still occupy SMs when sequences are long (Dao *et al.*, 2023). At moderate context, weight traffic and per-layer launch overhead still compete with attention IO depending on model width and hardware class (Chen, 2026a; Vellaisamy *et al.*, 2026). Your worksheet should include at least two context cells; optimizing only ctx=512 ships surprises at production context.

1.5 Why Mainstream Frameworks Do Not Solve Decode Alone

PyTorch plus Hugging Face is logically complete for research: every transformer sub-block is an op. For production decode it is physically naive—the *schedule* is chosen at runtime by a Python loop, not at compile time against memory hierarchy limits. Each boundary materializes tensors to HBM (or DRAM) that a fused schedule could keep in registers, shared memory, or cache.

Eager PyTorch loop (what goes wrong). A naïve `transformers` decode loop—one Python forward per token, eager ops for layer norm, QKV, attention, MLP, residuals—mirrors the math graph but not the memory hierarchy (Vellaisamy *et al.*, 2026). Each op exports intermediates to HBM; the next op imports them again. Ten to twenty boundaries per layer times tens of layers times hundreds of tokens per session is why launches and bytes both explode without fusion (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Serving frameworks wrap that loop with better batching; they do not automatically rewrite it into a residency-first schedule (Kwon *et al.*, 2023).

Three optimization layers (do not conflate). Serving framework (vLLM paging, continuous batching): better fleet utilization; same per-step kernel count inside the model (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). **Attention-kernel IO** (FlashAttention, Flash-Decoding): lower attention bytes; reported ≈8× *attention-kernel* speedups at long context, not end-to-end chat latency (Dao *et al.*, 2023). **Compiler fusion** (MegaKernel / YiRage): collapse layer boundaries; the subject of Parts V–VI.

On CPUs, fragmentation becomes LLC misses; on AMD XDNA-class NPUs, dynamic per-op dispatch violates static AI Engine Dataflow (ADF) graphs—the programming model for tile DMA and on-chip buffer residency (AMD, 2024b,a). There is no single "GPU recipe" to photocopy.

SLA vocabulary. Product documents often specify p50/p99 time-to-first-token (TTFT) separate from per-token decode latency. TTFT is dominated by prefill and queueing; streaming smoothness after the first token is dominated by decode (Fan, 2025; Li, 2024). A team that optimizes only TTFT can still lose users to stuttery streams when decode ms/token is high (Kwon *et al.*, 2023). Instrument both phases and both tail metrics; the worksheet in §1.7 applies to decode steps after the prompt is ingested.

Cost model sketch. Let C be GPU-hour cost, λ concurrent decode sessions, and $\bar{t}$ mean ms/token. Roughly, sustained decode throughput scales with $1/\bar{t}$ for memory-bandwidth-bound stacks; lowering bytes/token or kernels/token shifts $\bar{t}$ and directly changes how many paid users fit on a node (Chen, 2026a; Vellaisamy *et al.*, 2026; Wang, 2025b). You do not need a perfect finance model—you need proof that your change moved a counter that executives already believe correlates with cost (Fan, 2025).

Anti-pattern: "we enabled flash, we are done." FlashAttention-family kernels solve a *subset* of bytes in the attention block (Dao *et al.*, 2022, 2023). They do not fuse MLP and norm boundaries, do not fix MoE routing storms, and do not remove Python dispatch (Vellaisamy *et al.*, 2026). Treat flash as one rung on the ladder in Figure 1.4, then measure what remains (Chen, 2026a; Chen and YiRage Contributors, 2026).

1.6 Measuring Decode Goodput

Workload taxonomy. Interactive chat (batch-1 streaming) is the strictest cell: every token pays full KV traffic and users feel tail latency directly (Fan, 2025). **Concurrent assistant** sessions amortize some host work via continuous batching but still run thin per-token forwards inside each active request (Kwon *et al.*, 2023). **Offline batch** jobs can raise batch size and hide launches; RAG pipelines with long contexts often revert to memory-bound decode when KV exceeds on-device residency (Li, 2024). **Edge voice** adds power caps and static-graph requirements on NPUs where Python dispatch may be disallowed (AMD, 2024a,b). When this book says "decode," we mean the autoregressive forward that appends one token per step unless a table states otherwise (Wang, 2025a).

Fregly's **goodput** asks how much *useful* work reaches users per dollar and per second (Davies *et al.*, 2025). For interactive decode we make that concrete with four counters—the book's regression gates in every chapter:

Track all four on the **same configuration cell**: model revision, precision, context length, batch policy, framework and driver versions (Vellaisamy *et al.*, 2026; Chen, 2026a). Product p99 ms/token and kernel-team TFLOPs only reconcile when these counters move together.

Table 1.2: Decode goodput counters: metric, business lever, and what to do first.

Counter	Measures	Business lever	If bad, try first
bytes/token	Weight+KV traffic	DRAM/HBM cost, concurrency	Quantized GEMMs, fusion
kernels/token	Host launches	ms/token, tail latency	Graphs, MegaKernel
HDBI	Host vs device time	CPU load, jitter	`torch.compile`, fusion
R_{floor}	Bandwidth efficiency	Cap-ex ROI on fast GPUs	Bytes vs launches split

1.7 Two-Step Bottleneck Diagnosis

Reporting discipline. Report decode latency as **median ms/token** at batch-1 unless noted, with warmup discarded and fixed context lengths (512, 2048, 8192) (Chen, 2026a). For fleet traces, separate per-request ms/token from per-scheduler-step ms when continuous batching merges users (Kwon *et al.*, 2023; Chen, 2026b). Publish bootstrap confidence intervals when comparing graph capture or fusion A/B tests (Chen, 2026a). Tail metrics (p95/p99) matter because GC pauses, compilation, and MoE routing variance surface in tails even when median kernels/token looks stable (Vellaisamy *et al.*, 2026; Fan, 2025).

Before you rewrite kernels, run this worksheet (TaxBreak + Memory-Floor) (Vellaisamy *et al.*, 2026; Chen, 2026a):

Step 1 — Orchestration or device? Record kernels/token and HDBI. If kernels/token is in the hundreds+ on a dense model, or HDBI $\to 0$, suspect launch storms and framework tax.

Cross-checking TaxBreak and Memory-Floor. TaxBreak is host-centric: it tells you whether framework translation, library front-ends, or launch floors dominate orchestration time (Vellaisamy *et al.*, 2026). Memory-Floor is device-centric: it tells you whether observed step time tracks analytic bandwidth floors or is inflated by fixed per-step overhead (Chen, 2026a). Use both: a low HDBI with low R_{floor} on H100 screams launches; a high R_{floor} on L4 screams bytes (Chen, 2026a; Vellaisamy *et al.*, 2026). Neither tool replaces end-to-end ms/token—they explain *why* ms/token resists your last kernel tweak.

Step 2 — Bytes or launches on device? Compute R_{floor}. High R_{floor} ($\approx$0.7+ on bandwidth-class GPUs): optimize bytes moved. Low R_{floor} on H100-class silicon: optimize launches (CUDA Graphs, fusion) *before* swapping attention implementations—Memory-Floor shows FlashAttention-2 eager can lose to SDPA

Table 1.3: Verified kernel fragmentation on H100 decode (BS=4, SL=2048, m=10). Source: TaxBreak (Vellaisamy *et al.*, 2026).

Model	**Kernels/token**	**Total launches**	**GPU util. (%)**
Llama-3.2-1B	847.5	8,475	58.9
Llama-3.2-3B	1,536.9	15,369	67.6
OLMoE-1B/7B	9,305.3	93,053	15.5
Qwen1.5-MoE-A2.7B	6,695.1	66,951	27.7

when attention is not the binder (Chen, 2026a).

CUDA Graphs on Qwen-2.5-7B (H100, ctx=2048, batch-1) remove ≈3.05 ms (≈21%) of eager median step time; on L4 the same trick yields only 1.028× because the GPU is already bandwidth-saturated (R_{floor}=0.81) (Chen, 2026a; NVIDIA, 2024).

MoE models deserve a separate row in your spreadsheet: GPU utilization in the teens–twenties percent with multi-second-per-token tails is often dispatch, not FLOPs (Vellaisamy *et al.*, 2026; Wang, 2025a).

Tensor cores and TMA (preview). Tensor cores execute MMA instructions on large tiles; decode batch-1 shrinks effective M to one, inflating setup overhead (SemiAnalysis, 2024). Hopper's TMA (Tensor Memory Accelerator) issues bulk copies from global memory into shared memory with descriptor hardware, reducing scalar address arithmetic in fused kernels (SemiAnalysis, 2024). Neither replaces fusion: they are tools inside a dataflow schedule that already minimized bytes and launches (Chen and YiRage Contributors, 2026). Part III develops legality rules; Chapter 1 warns against TC-first tuning when worksheets say dispatch-bound.

1.8 Multi-Hardware Pain Points: GPU, CPU, and NPU

AMD GPU note. HIP stacks mirror CUDA concerns but differ in wavefront width, LDS (local data share) capacity, and ROCm graph support. Decode on AMD hardware still rewards fusion and launch reduction first; tensor-core analogs only help when GEMM dimensions justify setup (AMD, 2024a).

The pain *shape* is universal; the remedy is not.

GPUs (NVIDIA/AMD). SIMT (single-instruction, multiple-thread) warps, shared memory tiling, and coalesced loads matter; on decode, launch reduction

Table 1.4: First-order decode levers by hardware class.

Class	Optimize first
NVIDIA GPU	Fusion, CUDA Graphs, KV bytes, shared mem tiling
AMD GPU	HIP fusion, LDS reuse, launch reduction
CPU	Cache/NUMA blocking, SIMD, coarse fusion
Edge NPU	Static ADF graphs, DMA pipelines, full-layer fusion

and fusion often beat another tensor-core micro-kernel. Tensor cores execute MMA (matrix multiply–accumulate) ops; at batch-1 the effective M dimension is tiny, so MMA setup can dominate (SemiAnalysis, 2024). Hopper's TMA (Tensor Memory Accelerator) offloads bulk copy descriptor work but still prefers transfers large enough to amortize setup.

CPUs (x86/ARM). Bandwidth is cache- and NUMA-shaped. SIMD (single-instruction, multiple-data) width and KV layout aligned to cache lines beat warp thinking. On dual-socket servers, the first lever is often KV layout and coarse fusion that keeps QKV+MLP in last-level cache for one token step—not CUDA occupancy heuristics copied verbatim.

Edge NPUs (XDNA/AIE). Static ADF graphs, DMA ping-pong, and compile-time buffer proof dominate; Python dispatch may be disallowed in firmware. LDS (local data share—AMD GPU on-chip scratch, analogous to CUDA shared memory) and NPU SRAM slots are the residency budget compiler passes must prove (AMD, 2024b,a).

1.9 Industry Baselines and What They Still Leave on the Table

Leaderboard prefill tok/s and attention-only speedups answer legitimate research questions, not the batch-1 ms/token chat SLO (Li, 2024; Kwon *et al.*, 2023). Flash-Decoding reports up to ≈8× *attention-kernel* gains at long context; vLLM raises fleet goodput with paging—yet MLP, norms, sampling, and thousands of launches per MoE token remain (Dao *et al.*, 2023; Li, 2025; Vellaisamy *et al.*, 2026).

Our open baseline ladder: PyTorch eager → SDPA/FlashAttention → Flash-Decoding → paging/graphs → int4 GEMMs → MegaKernel/YiRage fusion. Each rung removes one bottleneck; regression reports speedup against the strongest rung

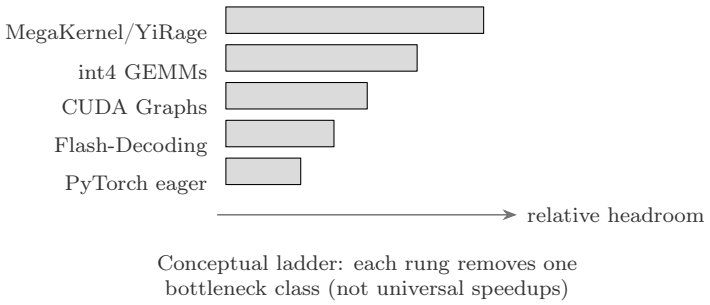


Figure 1.4: Open baseline ladder vs. compiler fusion target (qualitative; measure on your cell).

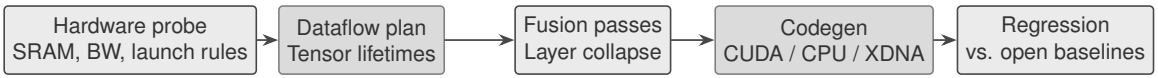


Figure 1.5: YiRage: probe hardware, plan residency, fuse layers, codegen per target, regress on goodput counters.

already enabled on that hardware (Chen, 2026a; Arya, 2024; Chen and YiRage Contributors, 2026).

Decode benchmarking pitfalls (public comparisons). Prefill-throughput masquerading as decode wins: tokens/s on long prompts without batch-1 ms/token omits what chat users feel (Li, 2024; Kwon *et al.*, 2023). **Attention-only speedups as end-to-end claims:** Flash-Decoding can accelerate attention dramatically while MLP, norms, sampling, and dispatch remain (Dao *et al.*, 2023; Chen, 2026a). **Mismatched batch policies:** comparing eager batch-1 microbenchmarks to continuous-batching server traces confounds orchestration with scheduling (Vellaisamy *et al.*, 2026; Chen, 2026b). Publish interactive cells alongside throughput headlines (Chen, 2026a; Wang, 2025a).

Benchmark hygiene. Publish at least one interactive cell: batch-1, fixed context, median *and* p99 ms/token, with kernels/token and bytes/token side by side (Chen, 2026a,b).

1.10 The YiRage Answer: Compiler-Scale Fusion (Preview)

YiRage is this book's reference **hardware-aware** LLM compiler: it ingests decoder graphs, models chip constraints, and emits fused schedules for CUDA, CPU, and XDNA-class targets (Chen and YiRage Contributors, 2026). It does not replace vLLM scheduling or FlashAttention math—it owns *what stays on chip* across layer boundaries.

Symptoms from this chapter become design requirements for later parts: **R1** minimize global round trips; **R2** prefer static schedules (especially NPUs); **R3** match fusion depth to hardware (GPU medium, CPU cache-aware, NPU layer-scale); **R4** amortize launches (graphs, persistent kernels, MegaKernels); **R5** gate every pass on kernels/token and bytes/token regressions (Appendix B).

1.11 Profiling Decode Without Foolish Optimism

Nsight Systems timelines are excellent for proving *who* waits on whom: Python framework threads, CUDA API gaps, and kernel bursts (NVIDIA, 2024). Nsight Compute kernel profiles are excellent for proving *whether* a kernel is bandwidth- or instruction-bound once launches are amortized. Use them in that order on decode cells: if the timeline shows launch storms, Compute profiles of individual GEMMs optimize the wrong bottleneck.

Cross-hardware normalization. When comparing GPU, CPU, and NPU targets, report paired ratios on identical model revisions *and* the four goodput counters (Chen, 2026a; AMD, 2024b; Chen and YiRage Contributors, 2026). A schedule that wins ms/token on H100 but fails XDNA buffer allocation is a bring-up failure, not a portable victory—Part VI encodes those outcomes as compiler errors rather than silent slowdowns (Chen and YiRage Contributors, 2026).

Practice implication. Before your next optimization sprint, freeze a configuration cell in a spreadsheet: model hash, precision, context, batch policy, framework and driver versions, kernels/token, bytes/token, HDBI, R_{floor}, median and p99 ms/token. Re-run the cell after every change; if only one column moves, document which bottleneck class you touched (Vellaisamy *et al.*, 2026; Chen, 2026a).

Compiler preview workflow (five steps). When you reach Part VI, YiRage-style compilation replays the same loop on every target: **(1) Hardware probe** — read bandwidth caps, SRAM sizes, launch rules; **(2) Global dataflow plan** — assign tensor lifetimes to register, shared/LDS, or global tiers; **(3) Fusion passes** — collapse decoder layers; insert async copy pipelines (TMA on Hopper, DMA on AIE); **(4) Codegen** — emit CUDA, CPU, or XDNA schedules with architecture-specific tile search; **(5) Regression** — compare against the strongest enabled open baseline on kernels/token and bytes/token (Chen and YiRage Contributors, 2026; Chen, 2020). Chapter 1 states the symptoms; later chapters show the passes.

What we do not claim. We do not promise a single universal speedup factor. We promise a measurement harness (Appendix B) and legality rules that fail loudly when residency cannot be proved (Chen and YiRage Contributors, 2026; Arya, 2024). That honesty is part of O'Reilly-style engineering writing: name the mechanism, name the counter, show the cell.

From symptoms to requirements (R1–R5 recap). **R1** minimize global round trips; **R2** prefer static schedules where hardware allows; **R3** shape fusion depth per target (GPU medium, CPU cache-aware, NPU layer-scale); **R4** amortize launches via graphs, persistent kernels, or MegaKernels; **R5** require counter movement on a fixed benchmark matrix before merging any pass (Chen and YiRage Contributors, 2026; Chen, 2020). These are not slogans—they are release gates YiRage encodes as hard failures when buffer residency or launch granularity cannot be satisfied (Chen and YiRage Contributors, 2026).

1.12 Book Roadmap and How to Read

Part III (ch04–05) is where mechanical sympathy becomes concrete: Hopper TMA and XDNA ADF are contrasting answers to the same question—how bytes enter on-chip SRAM before MMA or vector ops fire (SemiAnalysis, 2024; AMD, 2024b). Part V (ch10–12) ships the fused decode cell and compares eager launches, CUDA Graph capture, and persistent MegaKernels—the execution modes your framework team will ask about in review (NVIDIA, 2024; Chen and YiRage Contributors, 2026). Part VI (ch13–21) is deliberately one lowering arc across MLIR, XLA, TVM, Triton, IREE, Glow, and Mirage so you can compare backends on identical models rather than reading nine product brochures (Chen, 2020). Part VIII (ch28–30) closes the loop with inference frameworks, YiRage runtime, and three-way co-design

Table 1.5: Eight-part learning path (Fregly-style goodput arc, compiler depth).

	Part	You will be able to
I	ch01–02	Diagnose decode; think dataflow-first
II	ch03	Read hardware constraint matrices
III	ch04–05	Map Hopper CUDA and XDNA legality
IV	ch06–09	Plan on-chip residency and sync
V	ch10–12	Build and ship MegaKernels / graphs
VI	ch13–21	Lower the same IR across MLIR/XLA/TVM/...
VII	ch22–27	Profile fleets, MoE, autotune
VIII	ch28–30	Co-design frameworks, runtime, compiler

Table 1.6: Literature-backed decode anchors (conditions vary; sources in column 3).

Metric	Value	Source / conditions
Decode:prefill nodes	10:1	DeepSeek-V3 (Davies *et al.*, 2025)
Dense kernels/step	≈844	Llama-3.2-1B, H200, BS1/SL512 (Vellaisamy *et al.*, 2026)
MoE kernels/token	9,305	OLMoE, H100, BS4/SL2048 (Vellaisamy *et al.*, 2026)
Launch floor ΔKT	4.707 μs	H100 null kernel (Vellaisamy *et al.*, 2026)
CUDA Graphs (H100)	1.259×	Qwen-2.5-7B, ctx2048 (Chen, 2026a)
R_{floor} H100 / L4	0.27 / 0.81	Qwen-2.5-7B, ctx2048 (Chen, 2026a)

between framework schedulers, compilers, and serving SLOs (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Read Parts I–II before jumping to CUDA tile advice. Carry the four goodput counters into every profiling session; if they do not move, the optimization did not matter. Skim Part VIII early if you own a serving stack—framework schedulers and compiler fusion interact at decode boundaries even before you write CUDA (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Appendix A cross-links the full checklist; use it before major releases the same way reliability teams use launch runbooks (Davies *et al.*, 2025).

1.13 Key Takeaways

- **Diagnose before you optimize.** Run the two-step worksheet on every new model SKU: HDBI and kernels/token expose orchestration limits; R_{floor} tells you whether bytes or launches bind on the device. If HDBI $\to 0$, tensor-core tile tuning is theater—collapse launches or fuse layers first (Vellaisamy *et al.*, 2026; Chen, 2026a). Teams that skip this step often ship quarterly GEMM tweaks while p99 latency flatlines.

- **Memory and dispatch beat peak FLOPs in chat.** Interactive decode sits on the memory-bandwidth roofline at batch-1; tensor-core utilization can be $\leq 1\%$ in limit studies when compute is already provisioned (Davies *et al.*, 2025; Yu, 2026). Practice order: reduce global round trips, then kernel launches, then MMA inner loops. Buying a faster SKU without changing the schedule frequently buys a lower R_{floor}, not a faster chat (Chen, 2026a).

- **Measure goodput, not leaderboard tok/s.** Split prefill and decode dashboards; publish batch-1 median *and* p99 ms/token alongside kernels/token and bytes/token (Kwon *et al.*, 2023; Chen, 2026b; Fan, 2025). Goodput is useful tokens per dollar at your real batching policy—if the four counters do not move, the optimization did not matter.

- **Respect layer boundaries in the stack.** vLLM solves fleet scheduling and KV fragmentation; FlashAttention solves attention IO; MegaKernel/YiRage fusion solves layer residency (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Conflating those layers sends teams to tune paging when the binder is launch count, or tune attention when bytes/token is already at the roof.

- **Adapt remedies to the silicon target.** CUDA Graphs may rescue orchestration-limited H100 cells while L4 remains bandwidth-bound ($R_{\text{floor}} \approx 0.81$) and NPUs require static ADF proofs (Chen, 2026a; AMD, 2024b; NVIDIA, 2024). Medium-grain GPU fusion heuristics copied to CPU or NPU frequently *hurt* latency.

- **Prefer skillful engineering over brute-force spending.** Once paging, flash kernels, graphs, and quant backends are enabled, compiler-scale dataflow planning attacks the remaining boundaries with R1–R5 regression gates (Chen and YiRage Contributors, 2026; Chen, 2020; Arya, 2024). That is the skill this book teaches—not another peak-FLOPs datasheet comparison.

1.14 Conclusion

Decode-phase LLM inference is a systems problem disguised as a modeling problem (Nakai, 2025). Industrial teams already bought the GPUs and enabled the famous kernels; the remaining gap is schedule ownership—how many times per token weights and activations cross the memory wall, and how many launches the host injects (Vellaisamy *et al.*, 2026; Chen, 2026a).

When you finish this chapter, you should be able to walk into a performance review with a filled worksheet and a prioritized backlog: if orchestration-limited, schedule graph capture and fusion experiments; if bandwidth-limited on a mid-tier GPU, schedule bytes/token work (quantized GEMMs, KV layout, flash attention where it actually binds); if both counters look healthy but SLOs fail, audit batching policy and measurement cells before touching kernels (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023). Chapter 2 teaches how to *design* schedules that keep tensors on chip; Chapter 1 teaches how to prove you need that design (Chen, 2020; Goodfellow *et al.*, 2016).

Chapter 2 replaces operator-at-a-time thinking with dataflow-first residency planning—the bridge from these counters to Hopper/XDNA craft and YiRage IR (Chen, 2020; Goodfellow *et al.*, 2016).

Closing the business loop. When you present decode work to leadership, translate counters to money and SLA: lower bytes/token ⇒ more concurrent chats per GPU; lower kernels/token ⇒ better p99 streaming; higher R_{floor} on bandwidth-class GPUs ⇒ quant and fusion investments paying off; healthy HDBI ⇒ host fleet rightsizing (Vellaisamy *et al.*, 2026; Chen, 2026a; Fan, 2025). Avoid slides that only show TFLOPs or kernel-local speedups without ms/token at the product batching policy.

Worksheet gate. Before changing a kernel, state your model, batch, context, and whether HDBI and R_{floor} classify the cell as orchestration- or bandwidth-limited. If you cannot, run TaxBreak and Memory-Floor first (Vellaisamy *et al.*, 2026; Chen, 2026a).

Hands-on next step. Open your current decode trace or profiler export and count launches for one token step (or read kernels/token from a prior TaxBreak-style study) (Vellaisamy *et al.*, 2026). If the number exceeds a few hundred on a dense model, your next experiment is graph capture or fusion—not a faster attention kernel (NVIDIA, 2024; Chen and YiRage Contributors, 2026). If bytes/token is at

the bandwidth roof (R_{floor} high), prioritize quantization kernels that actually read narrow weights end-to-end (Chen, 2026a). Write the before/after counters in the spreadsheet template from Appendix F before requesting more hardware budget (Vellaisamy *et al.*, 2026; Chen, 2026a).

Glossary anchors (repeat for memory). **KV cache** stores prior layer keys and values so each new token reuses past context without recomputing the prompt. **Kernel launch** is the host CPU's submission of one GPU function; thousands per token create synchronization and framework overhead (Vellaisamy *et al.*, 2026). **HBM** is off-chip GPU memory with high bandwidth but high latency compared to registers and shared memory. **SIMT** (single-instruction, multiple-thread) is NVIDIA's warp execution model; **SIMD** is the CPU vector analogue—both matter when mapping fusion depth (SemiAnalysis, 2024; AMD, 2024b). **LDS** (local data share) is AMD GPU on-chip scratch memory, analogous to CUDA shared memory (AMD, 2024a). **ADF** (AI Engine Dataflow) is AMD's static graph model for XDNA tiles (AMD, 2024b).

Relationship to Chapter 2. Chapter 2 explains *how* to think in dataflow: which tensors must live on chip, for how many ops, and under which hardware-specific tile constraints (Chen, 2020; Goodfellow *et al.*, 2016). Chapter 1 explains *why* that mindset is non-optional for decode: operator-at-a-time schedules measurably pay launches and HBM round trips you can avoid (Vellaisamy *et al.*, 2026; Chen, 2026a). Read them as a pair—metrics first, design second (Chen and YiRage Contributors, 2026).

Appendix pointers. Appendix B hosts the reproducible benchmark harness; Appendix F hosts blank TaxBreak/Memory-Floor worksheets (Vellaisamy *et al.*, 2026; Chen, 2026a). Use them as living documents per model release—not one-off slides for a single executive review (Fan, 2025).

What success looks like. Success in this chapter is not memorizing roofline algebra—it is refusing to optimize blind. You can explain to a colleague, with counters in hand, whether your decode cell is orchestration-limited or bandwidth-limited, which stack layer owns the next experiment, and which open baseline you must beat (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023). That clarity is the mechanical sympathy Fregly advocates: software that respects how the machine actually spends time (Davies *et al.*, 2025). If you can fill Appendix F for your deployment today, you are ready for Chapter 2's dataflow design rules and

for the hardware chapters that follow (Chen, 2020; Chen and YiRage Contributors, 2026). Treat that worksheet as the entry ticket to every later optimization chapter in this book—no counter movement, no merge (Vellaisamy *et al.*, 2026; Chen, 2026a).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 6, 10, 11, 13, 14, 21

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 3, 9, 10, 11, 14, 16, 17, 19, 21

Arya, X. (2024). A preliminary performance analysis of llm inference on edge accelerators. 15, 17, 19

Chen, J. (2026a). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 2, 3, 4, 6, 7, 9, 11, 12, 13, 15, 16, 18, 19, 20, 21, 22

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 3, 9, 17, 19, 20, 21, 22

Chen, X. (2026b). Towards high-goodput llm serving with prefill-decode multiplexing. 5, 9, 12, 15, 19

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 2, 3, 4, 8, 9, 10, 11, 13, 15, 16, 17, 18, 19, 20, 21, 22

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 6, 9, 10, 11, 14, 15

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 2, 3, 9, 10, 11, 19

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 4, 5, 6, 7, 9, 11, 18, 19, 21

Eto, X. (2025). Llm performance bottlenecks on cgla. 5

Fan, X. (2025). Ellie: Energy-efficient llm inference at the edge via prefill-decode splitting. 2, 4, 6, 10, 11, 12, 19, 20, 21

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 20, 21

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 2, 3, 9, 10, 11, 12, 14, 15, 18, 19, 20, 21

Lazuka, X. (2024). Llm-pilot: Characterize and optimize performance of your llm inference services. 7

Li, X. (2024). Llm inference serving: Survey of recent advances and opportunities. 2, 5, 10, 11, 14, 15

Li, X. (2025). Llm-powered automated detection and optimization of inference bottlenecks in distributed systems: A static analysis approach. 14

Nakai, X. (2025). Efficient batch processing for private cloud llm inference: Modeling and performance comparison. 20

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 13, 16, 17, 19, 20

Pichlmeier, X. (2024). Performance characterization of expert router for scalable llm inference. 5

Qiao, X. (2026). Tellme: An efficient end-to-end ternary llm prefill and decode accelerator with table-lookup matmul on edge fpgas. 5

Recasens, X. (2025). Mind the memory gap: Unveiling gpu bottlenecks in large-batch llm inference. 2, 3, 5, 6

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 5, 13, 14, 17, 21

Tang, X. (2026). Tpla: Tensor parallel latent attention for efficient disaggregated prefill & decode inference. 6

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 2, 4, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22

Wang, X. (2025a). Decoupled analysis of dvfs effects in prefill and decode stages of large language model inference. 8, 11, 13, 15

Wang, X. (2025b). Pdacc: A 541 tokens/s@llama-1.2b effective throughput prefill-decode disaggregated mcm-based accelerator for multi-task large language model applications. 1, 2, 9, 11

Yu, X. (2026). Precision bottlenecks and sensitivity analysis in llm quantizationllm quantization bottlenecks. 5, 6, 19

Index

Decode phase, 3

Goodput, 3

LLM inference, 3

Prefill phase, 3